

CityScan — מדריך המערכת

כל תיקייה, כל קובץ, זרימת הבקשה, ואיך שמירה למסד הנתונים באמת מתבצעת

עודכן: 14 באוגוסט 2026 • מבוסס על `main` @ `456894a`
מחליף את `CityScan-Guide-HE.pdf` ואת `cityscan-guide.html`

תוכן

- | | |
|-------------------------------|-------------------------------|
| 1. מה המערכת עושה | 13. errors, config, container |
| 2. שלושת השירותים | 14. MongoDB ו-Mongoose |
| 3. מפת התיקיות | 15. הישות Hazard, שדה אחר שדה |
| 4. עקרון השכבות | 16. הכפילות הגאוגרפית |
| 5. routes — מיפוי כתובות | 17. האינדקסים |
| 6. middleware — מה שקורה לפני | 18. מסע של דיווח אחד |
| 7. HTTP — controllers בלבד | 19. ה-hook והמלכודת |
| 8. boundaries — חוזה הרשת | 20. תמונות — למה לא במסד |
| 9. logic — הלוגיקה העסקית | 21. Users ו-Logs |
| 10. converters — הגבול | 22. ה-Frontend |
| 11. repositories — השאילות | 23. מצב הדמו |
| 12. data — הישויות | 24. הכלים |
| | 25. הרצה מקומית |

חלק ראשון

מבט על

מה נבנה, ממה הוא מורכב, ואיפה כל דבר יושב.

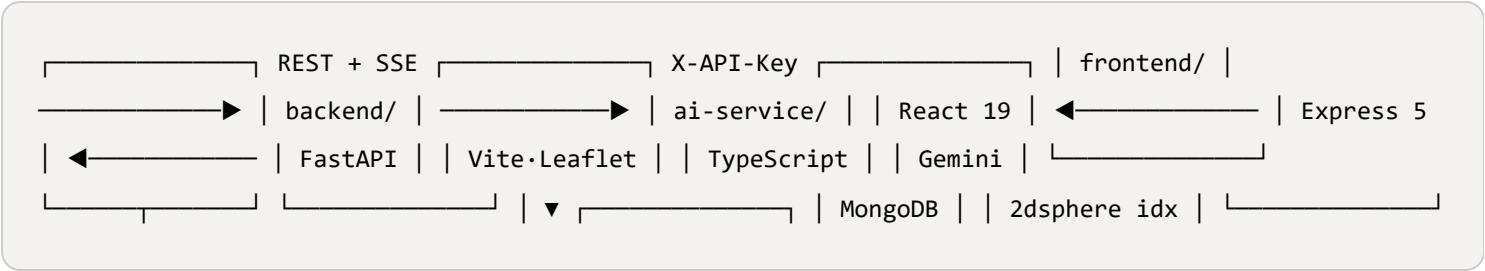
01 מה המערכת עושה

CityScan הופכת כל תושב לחיישן עירוני. אדם רואה מפגע ברחוב — בור בכביש, פנס שרוף, פסולת בנייה על המדרכה, הצפה — פותח את המפה, לוחץ על המקום, ומדווח. כל הדיווחים מופיעים על מפה משותפת שהעירייה רואה.

הבעיה שהמערכת באמת פותרת אינה הדיווח עצמו אלא **הכפילויות**. אותו בור בדיזנגוף מדווח אחת עשרה פעמים, ולתושב אין דרך לדעת שכבר דיווחו עליו. CityScan עונה על זה בשתי רמות:

- **רמה 1 — דטרמיניסטית.** מפגע לא-פתור מאותו סוג בתוך 50 מטר הוא כפילות, נקודה. שאילתה גאוגרפית שרצה במילישניות, בלי רשת ובלי תלות חיצונית.
- **רמה 2 — שיפוט.** רצה רק כשיש בסביבה מפגע מסוג אחר, שם השאלה באמת מעורפלת: "פסולת" ו"הצפה" במרחק חמישה מטרים עשויים להיות אותה בעיה פיזית או לא. את זה מכריע LLM.

רמה 2 נכשלת פתוח בכוונה. אם שירות ה-AI לא זמין, הדיווח עובר. תושב חייב תמיד להיות מסוגל לדווח, גם כשתלות אופציונלית למטה. זו גם הסיבה שזיהוי הכפילויות לא הועבר לכלי אוטומציה חיצוני.



שירות	שפה	למה דווקא היא
frontend/	TypeScript + React 19	מפה אינטראקטיבית עם מצב מורכב בצד הלקוח
backend/	TypeScript + Express 5	אותה שפה כמו ה-Frontend; טיפוסים משותפים בין השניים
ai-service/	Python + FastAPI	המקום שבו ספריות ה-AI חיות באמת

ההפרדה אינה קישוט. ה-Backend הוא היחיד שמחזיק את מפתח Gemini — הדפדפן לעולם לא רואה אותו, כי כל דבר שנשלח לדפדפן הוא פומבי. ה-Backend קורא ל- ai-service עם X-API-Key משלו.

CityScan/	
└ backend/src/	בדיקות 46 • Express 5 • קבצים 74
└ └ routes/	מיפוי כתובות + שרשור שומרים
└ └ middleware/	אימות, ולידציה, תפקידים, הגבלת קצב, שגיאות
└ └ controllers/	בלי בחירת סטטוס, try/catch בלבד – בלי HTTP
└ └ boundaries/	Zod חוזה הרשת + סכמות – DTO
└ └ logic/	ממשקי שירות – מה המערכת יודעת לעשות
└ └ └ impl/	המימושים – כל הלוגיקה העסקית כאן
└ └ converters/	Entity ⇌ Boundary
└ └ repositories/	שאלות בלבד
└ └ data/	נתוני זריעה, enums, Mongoose סכמות
└ └ errors/	שלהן HTTP-חריגות שנושאות את קוד ה
└ └ config/	באתחול Zod-פרופילי סביבה, מאומתים ב
└ └ scripts/	זריעה ותחזוקה
└ └ container.ts	שורש ההרכבה – הזרקת תלויות ידנית
└ └ app.ts	Express הרכבת
└ └ └ server.ts	נקודת הכניסה
└	
└ frontend/src/	
└ └ core/	config • http • session • demoVault
└ └ services/	authService • hazardService • aiService
└ └ pages/	Login • Register • Admin • MyReports
└ └ components/	MapComponent • ReportSidebar • NavBar • DemoBanner
└ └ contexts/	AuthContext
└ └ data/	demoFixtures
└ └ └ public/markers/	מאוחסנים אצלנו, PNG אייקוני 8
└	
└ ai-service/app/	FastAPI + Gemini
└ docs/	המסמך הזה
└ .github/workflows/	איפוס הדמו • CI

המבנה של `backend/src` ושל `frontend/src` אינו שרירותי — הוא מועתק מפרויקט Java קודם, SmartCollect, כדי שאותה ארכיטקטורה תיקרא זהה בשתי שפות. ב-SmartCollect ה-Frontend יושב ב-`src/main/resources/static/js` ומחולק ל-`core/`, `services/` ו-`pages/` — בדיוק מה שיש כאן.

ה-Backend, שכבה אחר שכבה

הכלל היחיד שמחזיק את הכל, ומה כל תיקייה עושה בפועל.

04 עקרון השכבות

כלל אחד מחזיק את המבנה כולו:

כל שכבה מכירה רק את זו שמתחתיה.

data/ לא יודעת ש- logic/ קיימת. logic/ לא יודעת ש-HTTP קיים — אין בה req, אין res, ואין מספרי סטטוס. זה מה שמאפשר לבדוק את הלוגיקה העסקית בלי לעלות שרת, ולהחליף את ספק ה-LLM בלי לגעת בה.

Request | ▼ routes/ ← אילו שומרים, איזו כתובת, | ▼ middleware/ → תפקיד → הגבלת קצב
| controllers/ ← מוציא נתונים מ req, מחזיר סטטוס, מקורא לשירות, | ▼ logic/impl/ ★ כל הכללים העסקיים ★
converters/ Entity → Boundary | repositories/ השאליות בלבד | ▼ data/ MongoDB סכמות | Response

שישה קבצים. כל אחד מגדיר אילו כתובות קיימות ואילו שומרים עוברים לפניהן. **מדיניות ההרשאות קריאה מהקבצים האלה לבדם** — אין צורך לפתוח את הבקר כדי לדעת מי רשאי לגשת לאן.

```
router.get('/admin/list',
  requireAuth,           // חייב טוקן תקף
  requireRole('admin'),  // חייב תפקיד מנהל
  validate(adminListQuerySchema), // הפרמטרים חייבים לעמוד בסכמה
  asyncHandler(ctrl.adminList) // ורק אז הבקר
);
```


קובץ	תפקיד
auth.middleware.ts	מאמת JWT, טוען את המשתמש ל- req
requireRole.ts	חוסם מי שאינו בתפקיד הנדרש
validate.ts	מריץ סכמת Zod על גוף/שאילתה/פרמטרים
rateLimiter.ts	מגביל קצב; מדלג בפרופיל test
demoRestrict.ts	חוסם כתיבה למנהל הדמו
asyncHandler.ts	עוטף פונקציות async כדי שחריגה תגיע לטיפול המרכזי
errorHandler.ts	המקום היחיד שכותב תגובת שגיאה

כל שגיאה יוצאת באותו מבנה, שנכתב במקום אחד בדיוק:

```
{
  "message": "A hazard of this type was already reported within 50m.",
  "status": 409,
  "timestamp": "2026-08-14T09:12:44.101Z",
  "path": "/hazards",
  "code": "DUPLICATE_HAZARD"
}
```

בפרופיל development נוסף גם ה-stack; בייצור הוא לא נשלח לעולם.

הבקרים דקים בכוונה. אין בהם `try/catch` ואין בחירה ידנית של קוד סטטוס.

```
create = async (req: Request, res: Response) => {  
  const hazard = await this.hazards.create(req.body, req.user!);  
  res.status(201).json(hazard);  
};
```

אם השירות זורק `ConflictException`, החרیגה נושאת את 409 בתוכה, `asyncHandler` מעביר אותה ל- `errorHandler`, וזה כותב את התגובה. הבקר לא צריך לדעת שכפילות קיימת.

DTO — האובייקטים שנוסעים ברשת. כל קובץ מייצא גם את סכמת ה-Zod שלו, והטיפוס נגזר ממנה:

```
export const createHazardSchema = z.object({
  type: z.enum(HAZARD_TYPES),
  latitude: z.number().min(-90).max(90),
  longitude: z.number().min(-180).max(180),
  description: z.string().trim().max(1000).optional(),
});

export type CreateHazardBoundary = z.infer<typeof createHazardSchema>;
```

למה `z.infer` ולא טיפוס נפרד: אם היה טיפוס שנכתב ביד וגם סכמה, הם היו נפרדים ביום שמישהו משנה אחד מהם. כאן הטיפוס נגזר מהסכמה — לא ניתן לשנות אחד בלי שהשני יזוז.

09

logic/ - logic/impl/ — הלוגיקה העסקית

logic/ מכיל **ממשקים**: מה המערכת יודעת לעשות. logic/impl/ מכיל את המימושים. זו ההפרדה שמאפשרת להחליף את Gemini בלי לגעת בכללים העסקיים.

ממשק	מימוש	שורות
hazards.service.ts	hazards.service.impl.ts	292
auth.service.ts	auth.service.impl.ts	109
ai.service.ts	gemini.ai.service.impl.ts	122
photoStorage.service.ts	localDisk.photoStorage.impl.ts	119
events.service.ts	emitter.events.service.impl.ts	37
demo.service.ts	demo.service.impl.ts	106

hazards.service.impl.ts הוא הלב. צורת כל מתודה זהה: **הרשה** → **החל את הכלל** → **שמור** → **המר ביציאה**, ובמקום לבחור קוד סטטוס — לזרוק חריגה מטיפוס מתאים.

ההמרה בין הישות שבמסד לבין ה-Boundary שיוצא ברשת. זה המקום שבו **גיבוב הסיסמה פיזית לא יכול לדלוף**, כי ה-Boundary נבנה שדה-שדה ואף פעם לא בהעתקה.

עושה שלושה דברים במקום אחד: `hazard.converter.ts`

1. **מנרמל את** `reportedBy`. Mongoose מחזיר לפעמים אובייקט מלא ולפעמים `objectId` חשוף, תלוי בשאילתה. הלקוח תמיד מצפה לאובייקט.

2. **הופך נתיבי תמונות ל-URL מלאים** כש- `PUBLIC_BASE_URL` מוגדר.

3. **לא מעביר את** `location`, **את** `__v`, **ואת כל מה שפנימי**. הם פשוט לא מגיעים ל-Boundary.

כל שאלתה נמצאת כאן ורק כאן. שירות מבקש "מפגעים פתוחים ברדיוס 50 מטר" ואף פעם לא לומד מה זה רדיאן.

סכמות Mongoose, ה-enum, ונתוני הזריעה. ה-enum הם ההגדרה היחידה של כל קבוצת ערכים — סכמת Mongoose, סכמת Zod וכל בדיקת ריצה קוראות מהם:

```
export const HAZARD_TYPES = [
  'pothole', 'broken_streetlight', 'debris', 'flooding', 'other',
] as const;
export type HazardType = (typeof HAZARD_TYPES)[number];
```

הוספת סוג מפגע חדש היא שינוי במקום אחד.

`errors/` — כל חריגה נושאת את קוד ה-HTTP שלה. `NotFoundException` היא 404, `ConflictException` היא 409. הלוגיקה זורקת משמעות, לא מספרים.

`config/` — `NODE_ENV` בוחר פרופיל, בסגנון `application-{profile}.properties` של Spring. סדר הטעינה: משתני סביבה אמיתיים ← `.env.{profile}` ← `.env`, הראשון מנצח.

	TEST	PRODUCTION	DEVELOPMENT	
מסד	בזיכרון, חד-פעמי	Atlas	Docker מקומי	
	נקבע ע"י הבדיקות	חובה, ≤ 32 תווים	אופציונלי, מזהיר	JWT_SECRET
CORS	—	חובה	פתוח	
גוף שגיאה	כולל stack	הודעה גנרית	כולל stack	

שום מודול לא קורא `process.env` בזמן ייבוא. הכלל הזה הוא מה ששומר על סדר הטעינה נכון — ב-ES Modules הייבואים נטענים לפני גוף המודול, כך שקריאה מוקדמת הייתה קוראת סביבה שעוד לא נטענה.

`container.ts` — שורש ההרכבה. Spring בונה את גרף התלויות מאנוטציות; כאן זה נעשה במפורש, פעם אחת, כך שכל הגרף קריא בקובץ אחד. מעבר מ-Gemini לספק אחר הוא שורה אחת. כאן.

איך שמירה למסד באמת עובדת

מהלחיצה על המפה ועד השורה ב-MongoDB — כל שלב, כל שדה, וכל מלכודת.

Mongoose-14 MongoDB

MongoDB הוא מסד מסמכים. במקום טבלאות עם עמודות קבועות, הוא שומר *מסמכים* בפורמט דמוי JSON, ואוסף מסמכים נקרא *collection*. אין `CREATE TABLE` ואין מיגרציות כדי להוסיף שדה.

Mongoose היא ספרייה שמוסיפה מעל זה *סכמה* — הגדרה של אילו שדות קיימים, מאיזה טיפוס, ומה חובה. MongoDB עצמו לא יאכוף כלום; Mongoose כן.

```
// backend/src/config/db.ts
await mongoose.connect(config.db.uri, {
  dbName: config.db.name,
  maxPoolSize: 10,           // עד 10 חיבורים במקביל
  serverSelectionTimeoutMS: 10_000, // שניות ואז כישלון מפורש 10
});
```

למה `maxPoolSize: 10`: פתיחת חיבור למסד יקרה. מאגר חיבורים שומר עשרה פתוחים וממחזר אותם, במקום לפתוח חיבור לכל בקשה. עשרה מספיקים בנוחות למכונה של 1GB.

למה `timeout` מפורש: בלעדיו בקשה מול מסד שנפל תלויה ללא הגבלה. עשר שניות ואז שגיאה קריאה.

שלושה `collections`: `logs`, `users`, `hazards`.

```
const hazardSchema = new Schema<HazardEntity>(
  {
    type: { type: String, required: true, enum: HAZARD_TYPES },
    latitude: { type: Number, required: true, min: -90, max: 90 },
    longitude: { type: Number, required: true, min: -180, max: 180 },
    location: {
      type: { type: String, enum: ['Point'], default: 'Point' },
      coordinates: { type: [Number], default: undefined },
    },
    description: { type: String, trim: true },
    address: { type: String, trim: true },
    hazardPhotos: { type: [String], default: undefined },
    status: { type: String, enum: HAZARD_STATUSES, default: 'open' },
    reportedBy: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  },
  { timestamps: true }
);
```

שדה	מה נשמר	הערה
<code>_id</code>	<code>ObjectId</code>	נוצר אוטומטית — 12 בתים הכוללים חותמת זמן
<code>type</code>	מחרוזת מתוך 5	נאכף ע"י Mongoose וגם ע"י Zod <code>enum</code>
<code>latitude</code>	מספר	טווח נאכף בסכמה, לא רק בוולידציה
<code>longitude</code>	מספר	—
<code>location</code>	GeoJSON Point	שכפול מכוון — ראה פרק 16
<code>description</code>	מחרוזת	מסיר רווחים בקצוות <code>trim</code>
<code>address</code>	מחרוזת	מגיע מ-Nominatim
<code>hazardPhotos</code>	מערך כתובות	לא התמונות עצמן — ראה פרק 20
<code>status</code>	<code>resolved</code> / <code>in_progress</code> / <code>open</code>	ברירת מחדל <code>open</code>
<code>reportedBy</code>	<code>User</code> → <code>ObjectId</code>	הפניה, לא העתק
<code>createdAt</code>	תאריך	נוצר ע"י <code>timestamps: true</code>
<code>updatedAt</code>	תאריך	מתעדכן בכל שמירה

`default: undefined` על `hazardPhotos` אינו זהה להשמטה. בלעדיו Mongoose היה כותב `[]` לכל מפגע בלי תמונות — שדה מיותר בכל מסמך.

16 הכפילות הגאוגרפית — ולמה היא מכוונת

המיקום נשמר **פעמיים**: פעם כ- `longitude / latitude` שטוחים, ופעם כאובייקט `location` בפורמט GeoJSON. זה נראה כמו טעות. זה לא.

MongoDB יודע לחפש לפי מרחק רק על שדה GeoJSON עם אינדקס 2dsphere. על שני מספרים רגילים אי אפשר לשאול "מה בתוך 50 מטר מכאן" — הוא היה חייב לסרוק כל מסמך ולחשב מרחק בעצמו.

ולמה לא לוותר על השדות השטוחים ולהשאיר רק GeoJSON? כי הלקוח, המפה, וכל קוד קריאה רוצים `h.latitude`. לחלץ אותו מ- `location.coordinates[1]` בכל מקום היה מעיק ומועד לטעויות.

המלכודת שחייבים לזכור: GeoJSON שומר `[longitude, latitude]` — קו אורך קודם. זה הפוך מהסדר שרוב האנשים כותבים. החלפה בין השניים לא זורקת שגיאה; היא פשוט מציבה את תל אביב באמצע סומליה.

```
this.location = { type: 'Point', coordinates: [this.longitude, this.latitude] };  
// אורך ראשון ^^^^^^^
```

17 האינדקסים — ומה כל אחד עושה

אינדקס הוא מבנה נתונים נפרד ששומר את הערכים ממוינים, כך שהמסד ימצא שורה בלי לסרוק את כולן — כמו מפתח עניינים בסוף ספר. בלי אינדקס, שאילתה על מיליון מסמכים קוראת מיליון מסמכים.

```
hazardSchema.index({ location: '2dsphere' });
hazardSchema.index({ status: 1, createdAt: -1 });
hazardSchema.index({ reportedBy: 1, createdAt: -1 });
```

משרת	אינדקס
שאילתות רדיוס. מחשב על כדור , לא על מישור — כלומר מרחקים נכונים על פני כדור הארץ	<code>location: '2dsphere'</code>
המפה ורשימת המנהל: סנן לפי סטטוס, מיין מהחדש לישן	<code>{status:1, createdAt:-1}</code>
"הדיווחים שלי"	<code>{reportedBy:1, createdAt:-1}</code>

1 הוא סדר עולה, -1 יורד. הסדר בתוך אינדקס מורכב חשוב: אינדקס על `{status, createdAt}` משרת שאילתה שמסננת לפי סטטוס וממיינת לפי תאריך; הפוך — לא.

איך נראית שאילתת הרדיוס בפועל

```
// backend/src/repositories/hazard.repository.ts
const EARTH_RADIUS_METERS = 6_378_100;

Hazard.find({
  status: { $in: UNRESOLVED_STATUSES },
  location: {
    $geoWithin: {
      $centerSphere: [[longitude, latitude], radiusMeters / EARTH_RADIUS_METERS],
    },
  },
})
```

`$centerSphere` מקבל את הרדיוס ב**רדיאנים**, לא במטרים — ומכאן החלוקה ברדיוס כדור הארץ. 50 מטר הופכים ל-`0.00000784` רדיאן. החישוב הזה קיים במקום אחד בדיוק בכל בסיס הקוד.

מה קורה מהרגע שאדם לוחץ "שלח" ועד שהשורה קיימת במסד. זה המסלול המלא.

[1] POST /hazards hazardService.ts { type, latitude, longitude, description, hazardPhotos: [base64] } | [2] routes/hazard.routes.ts | requireAuth → נעצר כאן 401, נעצר כאן → [3] controllers/hazard.controller.ts | await this.hazards.create(req.body, req.user) ▼ [4] logic/impl/hazards.service.impl.ts ★ | ★ כאן כל הכללים → ?ברדיוס 50 מ' | — אותו סוג נמצא \$geoWithin → repositories → a. assertNotDuplicate() | ↳ repositories → \$geoWithin נמצא סוג אותו 4 | a. assertNotDuplicate() | ↳ repositories → \$geoWithin נמצא סוג אותו 409 X ConflictException | לא זמין? ממשיכים ai-service → נעצר | — סוג אחר בסביבה → ממשיכים | 4 | WebP → אקסקטנה ל-1600 EXIF סיבוב sharp: ↳ photos.saveMany() | b. כלום בסביבה? → ממשיכים | 4 | רק עכשיו נוגעים במסד ▼ [5] 4c. hazards.create() ← [5] repositories/hazard.repository.ts | Hazard.create(data) ▼ [6] data/hazard.entity.ts | pre('save') → location = { type:'Point', coordinates:[lng, lat] } | timestamps → createdAt, updatedAt | enum/min/max ▼ נאכפים MongoDB | db.hazards.insertOne({ ... }) | | קורא מחדש עם repository שלושת האינדקסים מתעדכנים | ↳ [7] populate('reportedBy') ▼ [8] converters/hazard.converter.ts | location ו-__v נתיב תמונה | לא עוברים → URL Created + HazardBoundary לכל הטאבים הפתוחים ▼ [10] 201 SSE 'hazard:created' → events מלא ▼ [9]

שימו לב לסדר בשלב 4. בדיקת הכפילות רצה לפני שנשמרת תמונה ולפני שנכתב משהו למסד. דיווח כפול לא מבזבז מקום בדיסק.

ומה אם השמירה נכשלת אחרי שהתמונות כבר נכתבו

```
let created: HazardEntity;
try {
  created = await this.hazards.create({ ... });
} catch (err) {
  if (storedPhotos?.length) await this.photos.removeMany(storedPhotos);
  throw err;
}
```

בלי זה, כל כישלון כתיבה היה משאיר קבצים על הדיסק שאף שורה במסד לא מצביעה אליהם — דליפה שגדלה לאט ואף אחד לא מבחין בה.

```
{
  "_id": ObjectId("66bd12f4a1c2e3d4f5a6b7c8"),
  "type": "pothole",
  "latitude": 32.0798,
  "longitude": 34.7739,
  "location": { "type": "Point", "coordinates": [34.7739, 32.0798] },
  "description": "בור עמוק בנתיב הימני",
  "address": "דיונוף 120, תל אביב",
  "hazardPhotos": ["/uploads/1755162412-a3f9c1.webp"],
  "status": "open",
  "reportedBy": ObjectId("66bc01aa9f8e7d6c5b4a3928"),
  "createdAt": ISODate("2026-08-14T09:06:52.411Z"),
  "updatedAt": ISODate("2026-08-14T09:06:52.411Z"),
  "__v": 0
}
```



```
hazardSchema.pre('save', function (this: HazardEntity) {
  if (this.latitude != null && this.longitude != null) {
    this.location = { type: 'Point', coordinates: [this.longitude, this.latitude] };
  }
});
```

ה-hook מבטיח ש- `location` תמיד תואם ל-`lat/lng`. אבל יש בו מגבלה שחייבים להכיר:

`pre('save')` **רץ רק על** `.save()`. אופרטורים אטומיים כמו `findOneAndUpdate` או `updateOne` עוקפים אותו לחלוטין. עדכון קואורדינטות דרכם היה משאיר את `location` על הערך הישן — והמפגע היה ממשיך להופיע בחיפושי רדיוס במקום הקודם שלו, בלי שום שגיאה.

לכן `HazardRepository.save()` מקבל מסמך מלא וקורא `doc.save()`, ולא מקצר דרך:

```
async save(doc: HazardEntity): Promise<HazardEntity> {
  await doc.save();
  const populated = await this.findById(doc._id.toString());
  return populated ?? doc;
}
```

המחיר: שתי פניות למסד במקום אחת. התמורה: `location` לא יכול לצאת מסנכרון.

איתו עיקרון בכיוון ההפוך: ה-seeder `κ` משתמש ב- `Hazard.collection.updateOne` — הדרייבר הנייטיב, מתחת ל- `Mongoose` — כי הוא צריך לתארך דיווחים אחורה, ו- `timestamps: true` היה דורס כל ניסיון לקבוע `createdAt` ידנית.

20 תמונות — למה הן לא במסד

בגרסה הראשונה התמונות נשמרו כ-base64 בתוך המסמך. שלוש בעיות:

1. **תקרת 16MB**. למסמך ב-MongoDB יש גודל מרבי. תמונת טלפון היא כ-1.5MB ב-base64; כמה תמונות לדיווח והתקרה מתקרבת.

2. **Atlas M0 החינמי מוגבל ל-512MB**. כמאה דיווחים והוא מלא.

3. **כל קריאת רשימה משכה מגה-בייטים**. המפה מושכת עד 500 מפגעים; התמונות נזרקו מיד.

הפתרון: `sharp` דוחס לקובץ WebP על הדיסק, ורק הכתובת נשמרת.

```
const webp = await sharp(buffer, { failOn: 'none' })
  .rotate() // EXIF כבד את דגל הכיוון של
  .resize({ width: 1600, height: 1600,
    fit: 'inside', withoutEnlargement: true })
  .webp({ quality: 80 })
  .toBuffer();

const filename = `${Date.now()}-${randomUUID()}.webp`;
```

1.5MB הופכים לכ-150KB — כעשירית. ובכל שאילתת רשימה, ה-repository מוציא את השדה מראש:

```
const WITHOUT_PHOTOS = '-hazardPhotos'; // המינוס = אל תחזיר את השדה הזה
```

הסרת המטא-דאטה היא לא רק חיסכון במקום. תמונות מטלפון נושאות EXIF ובו **קואורדינטות GPS**, והתמונות האלה מוצגות בפומבי על מפה. הדיווח כבר נושא קואורדינטות מפורשות; אין שום סיבה לפרסם בנוסף את הנקודה המדויקת שבה המדווח עמד. `sharp` משמיט את כל המטא-דאטה אלא אם קוראים ל-`withMetadata()` — ולא קוראים. `.rotate()` חייב לרוץ לפני שהמטא-דאטה נזרקת, אחרת תמונות שצולמו לאורך יופיעו מסובבות.

```
const userSchema = new Schema<UserEntity>({
  email:    { type: String, required: true, unique: true, trim: true, lowercase: true },
  password: { type: String, required: true, minlength: 6, select: false },
  name:     { type: String, trim: true },
  role:     { type: String, enum: USER_ROLES, default: 'user' },
}, { timestamps: true });
```

מאפיין	מה הוא מונע
unique: true	שני חשבונות לאותו מייל. נאכף ע"י אינדקס ייחודי במסד
lowercase: true	אלון@x.com -I Alon@x.com אלון@x.com כשני משתמשים
select: false	קריאה רגילה לא טוענת את הגיבוב כלל

select: false הוא ההגנה הראשונה: find() רגיל לא מחזיר את הסיסמה, ושאילתה שכן צריכה אותה חייבת לבקש במפורש (select('+password')). ההגנה השנייה היא user.converter.ts, שבונה את ה-Boundary שדה-שדה — כך שגם אם הגיבוב נטען, אין לו לאן לדלוף.

הסיסמה עצמה נשמרת כגיבוב bcrypt, לעולם לא כטקסט.

logs — שובל ביקורת. userId, action, resource, resourceId, details, ושלושה אינדקסים לפי דפוסי השאילתה.

חלק רביעי

Frontend-ה

אותו שלד כמו ב-SmartCollect, בטיפוסים.

תוכן	CITYSCAN	SMARTCOLLECT
IS_DEMO , AI_SERVICE_BASE , API_BASE	core/config.ts	core/config.js
api , getAuthHeader , apiFetch	core/http.ts	core/http.js
localStorage טוקן ומשתמש ב-	core/session.ts	core/session.js
מחסן הדמו	core/demoVault.ts	—
התחברות, הרשמה	services/authService.ts	services/userService.js
כל קריאות המפגעים + הטיפוסים	services/hazardService.ts	services/objectService.js
ניתוח תמונה	services/aiService.ts	services/commandService.js

שלושה כללים הועתקו מהמקור:

1. **core/ לא יודע כלום על הדומיין.** ולכן UNRESOLVED , DUPLICATE_RADIUS_METERS -I distanceMeters יושבים ב-hazardService.ts ולא ב-config.ts . core/config.ts לא יודע מה זה בור.
2. **שם השירות = שם ה-boundary בשרת.** UserBoundary → userService , ולכן authService , hazardService , aiService .
3. **שירות מחזיק את כל הדומיין שלו,** לא רק קריאות HTTP — כמו ש-objectService.js מחזיק גם buildBin וגם binTypeColor .

components/ -I contexts/ הן התוספת היחידה מעבר לשלד, כי React דורש אותן. AuthContext דק בכוונה: הוא לא מחזיק מפתחות אחסון ולא נוגע ב-localStorage בעצמו — core/session הוא הבעלים היחיד — הוא רק משקף אותו ל-state כדי שהעץ ירונדר מחדש בהתחברות וביציאה.

הפריסה הציבורית ב-Vercel רצה בלי Backend כלל. `VITE_IS_DEMO=true` מפעיל את `core/demoVault.ts` : כל כתיבה הולכת ל- `localStorage` וכל קריאה חוזרת משם, זרוע מ-15 המפגעים שב- `data/demoFixtures.ts`.
הזריעה מתרחשת רק כשהמפתח נעדר, לא כשהוא מפרש לקבוצה ריקה. מבקר שמחק את כל המפגעים משאיר רשומות עם ערך `null` — המפתח קיים, ואסור לזרוע מחדש, אחרת מחיקה הייתה נראית כאילו לא עשתה כלום.

יכולת	בדמו
דיווח, עריכה, מחיקה	עובד, נשמר בדפדפן
חסימת כפילות — רמה 1	עובד, מחושב בדפדפן
חסימת כפילות — רמה 2	לא זמין — דורש מפתח LLM
תיאור תמונה ב-AI	לא זמין — אותה סיבה

למה רמה 2 לא יכולה לרוץ בדפדפן: כל משתנה שמתחיל ב- `VITE_` מוטמע לתוך ה-`JavaScript` שנשלח. מפתח API בחבילת Vite הוא מפתח מפורסם. לכן הדמו אומר את זה במפורש במקום לירות בקשות לשרת שאינו קיים ולהיראות תקוע.

חלק חמישי

הכלים

מה כל אחד עושה, כמה הוא עולה, ולמה נבחר.

Vercel — אירוח ה-Frontend

פלטפורמה שמארח אתרים סטטיים. מחוברת ל-GitHub: כל דחיפה ל- `main` בונה ומפרסת אוטומטית. **חינם** בתוכנית Hobby.

נדרש `vercel.json` אחד, כי CityScan היא SPA — ניווט ל- `/login` מטופל ב-JavaScript, אבל טעינה ישירה של הכתובת מבקשת מהשרת קובץ שאינו קיים. בלי ההפניה, כל כתובת פנימית מחזירה 404:

```
{ "rewrites": [{ "source": "/*", "destination": "/index.html" }] }
```

Instant Rollback מחזיר את הפריסה הקודמת מיידית, בלי git ובלי בנייה מחדש. זו רשת הביטחון האמיתית.

Azure — אירוח ה-Backend

ענן של Microsoft. Azure for Students נותן **\$100 קרדיט בלי כרטיס אשראי** ובנוסף **750 שעות חודשיות של מכונה burstable בחינם ל-12 חודשים** — משרה מלאה, כך שהקרדיט כמעט לא נגמס.

שתי נקודות שנבדקו ומשנות את התכנון:

- B1s** מיועד לגריעה ב-2028 ו-Microsoft כבר חוסמת יצירה שלו במנויי סטודנטים רבים. התחליף הוא `Standard_B2ats_v2` — מכוסה באותן 750 שעות, ולמעשה חזק יותר: 2 vCPU במקום 1. חשוב לבחור את גרסת ה-AMD (`B2ats`) ולא את גרסת ה-ARM (`B2pts`), אחרת צריך לבנות מחדש את כל ה-Docker images ל-`arm64`.
- כתובת IPv4 ציבורית אינה חינם** — כ-`$3.65` לחודש, כ-`$44` לשנה. זו ההוצאה היחידה, ויש מקום בקרדיט.

Docker — אריזה

אורז אפליקציה יחד עם כל התלויות שלה לקונטיינר שרץ זהה בכל מקום. פותר את "אצלי זה עבד".

הבדל מ-SmartCollect: שם Docker Desktop הריץ את **מסד הנתונים** בזמן פיתוח, והאפליקציה רצה מחוץ לו. כאן הוא ירוץ בייצור ויארז את **האפליקציה עצמה** — `ai-service` ו-`backend` ו-`proxy`, שלושה קונטיינרים על אותה מכונה.

MongoDB Atlas — המסד המנוהל

MongoDB כשירות. רובד **M0 חינם לנצח** — 512MB. מספיק בנוחות מרגע שהתמונות עברו לדיסק. גיבויים וניטור כלולים.

HTTPS — דומיין DuckDNS + Nginx Proxy Manager

DuckDNS נותן תת-דומיין חינם (`cityscan.duckdns.org`) שמצביע לכתובת ה-IP של השרת.

Nginx Proxy Manager הוא *reverse proxy* — שרת שיושב בחזית, מקבל את כל התעבורה, ומפנה כל בקשה לקונטיינר הנכון. הוא גם משיג תעודת HTTPS מ-Let's Encrypt אוטומטית ומחדש אותה לבד.

הגדרה אחת שאסור לשכוח: `proxy_buffering off` על `/hazards/stream` . בלעדיה nginx צובר את אירועי ה-SSE בבאפר וממתין שיתמלא — והאירועים לא מגיעים ללקוח לעולם.

CI — GitHub Actions ותזמון

מריץ קוד אוטומטית באירועי מאגר. **חינם וללא הגבלה במאגר ציבורי**. שני workflows:

- `ci.yml` — בכל דחיפה: בונה ובודק את ה-Backend (46 בדיקות), בונה את ה-Frontend, ומוודא שה-ai-service נטען.
- `demo-reset.yml` — מחזיר את הדמו למצב הזרוע. **התזמון מושבת כרגע** ורק הרצה ידנית פעילה, כי אין עדיין Backend לאפס. הוא נכשל כל לילה ארבע פעמים עד שהוסר.

n8n — ולמה הוא לא בייצור

כלי אוטומציה חזותי: בונים תהליכים מגרירת בלוקים במקום לכתוב קוד. **הוחלט לא להשתמש בו בייצור, משתי סיבות נפרדות:**

1. **זיכרון.** למכונה יש n8n 1GB לבדו רוצה כ-400MB. לא היה נשאר מקום ל-backend, ל-ai-service, ל-proxy ולמערכת ההפעלה. האיפוס הלילי עבר ל-GitHub Actions, שחינמי ולא צורך מהשרת כלום.
 2. **ארכיטקטורה.** זיהוי הכפילויות נשאר ב- `logic/` ולא הועבר ל-n8n, כי הוא על הנתיב הקריטי — המשתמש ממתין ל-201 או 409. העברה לכלי חיצוני הייתה אומרת שאי אפשר לדווח כשהכלי למטה.
- אפשר להריץ אותו מקומית אם רוצים אותו לסיפור הפורטפוליו.

LLM — Gemini

המודל של Google. משמש לשני דברים: הכרעת כפילויות ברמה 2, ותיאור אוטומטי של תמונת מפגע. הרובד החינמי מספיק בקלות.

שיעור מהדרך: `gemini-1.5-flash` נגרע והחזיר 404. אחר כך גם `gemini-2.5-flash` החזיר 404 — **למרות שהופיע ברשימת המודלים**, כי הוא כבר לא נפתח למשתמשים חדשים. הרשימה אינה מקור אמת; חייבים לבדוק בפועל.

המפה — Leaflet · CARTO · Nominatim

- **Leaflet** — ספריית המפות. חינם, בלי מפתח, בלי מכסה.
- **CARTO Voyager** — אריחי המפה. אותו מידע כמו OpenStreetMap בפלטה מאופקת, כדי שסמני המפגעים יבלטו במקום להתחרות בצבעי הכבישים.
- **Nominatim** — המרה בין כתובת לקואורדינטות ולהפך. חינם, בלי מפתח.

Google Maps נשקל ונדחה: דורש חשבון חיוב, ומאז מרץ 2025 זה 10,000 טעינות בחודש ואז כ-7\$ לכל 1,000. כשהמכסה נגמרת המפה מתה עד חצות.

שמונת אייקוני הסמנים **מאוחסנים אצלנו** ב- `frontend/public/markers/`. קודם הם נמשכו מ- `raw.githubusercontent.com` — 28 תמונות משרתים זרים בכל טעינה. הכתובת ההיא **אינה CDN**: היא מגישה קבצים גולמיים לצורך עיון, ו-GitHub מגבילה עליה קצב. ביום שזה היה קורה, כל סמן על המפה היה נעלם בלי שאיש נגע בקוד.

הספריות שעושות את העבודה

ספרייה	תפקיד
<code>zod</code>	ולידציה + גזירת טיפוסים מאותה הגדרה
<code>mongoose</code>	סכמות, אינדקסים ו- <code>hooks</code> מעל MongoDB
<code>sharp</code>	דחיסה ל-WebP והסרת EXIF
<code>jsonwebtoken</code>	JWT — טוקן חתום שמוכיח מי המשתמש
<code>bcrypt</code>	גיבוב סיסמאות, איטי בכוונה
<code>express-rate-limit</code>	הגבלת קצב
<code>vitest</code> + <code>mongodb-memory-server</code>	46 בדיקות מול <code>mongod</code> אמיתי שנוצר מחדש בכל הרצה

חלק שישי

הרצה

```
# התקנה
npm install --prefix backend
npm install --prefix frontend
npm install

# מסד מקומי – פעם אחת אחרי כל אתחול של המחשב
npm run db:up --prefix backend
npm run seed:demo --prefix backend

# שלושת השירותים
npm run dev
```

פתח `http://localhost:5173` ולחץ **Sign in as Admin (Demo)** — בלי הרשמה.

כדי לראות את חסימת הכפילויות: לחץ על המפה ברחוב דיזנגוף במקום שכבר יש בו סמן, ודווח על **בור** נוסף. הדיווח נדחה.

הרצת ה-FRONTEND כמו בפריסה החיה

```
npx cross-env VITE_IS_DEMO=true npm run dev
```

זו התצורה המדויקת ש-Vercel מריץ. **בנייה נקייה אינה מוכיחה שהיא עובדת** — צריך להריץ ולבדוק: התחברות דמו, 12 סמנים, חסימת הכפילות בדיזנגוף, פאנל הניהול, ואפס שגיאות בקונסולה.

הבדיקות

```
npm test --prefix backend
```

46 בדיקות אינטגרציה מול `mongod` אמיתי שנוצר מחדש לכל הרצה דרך `mongodb-memory-server` — בלי Docker, בלי נתונים משותפים בין מקרים.

המסמך הזה נכתב מול הקוד ולא מהזיכרון. כל קטע קוד כאן הועתק מהקובץ שצוין לידו במצב `main` @ `456894a`.